SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1023	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	SIVAKUMAR M	**Reg. No.:**	192521057
Date:	07/10/26	**Duration:**	60 minutes

Code of Conduct

I SIVAKUMAR M(192521057)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: SIVAKUAMR M

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement**
   - Explain the problem in your own words.
   - Identify the objectives of the proposed system.
   - State the scope of the problem.
2. **Identify Key Stakeholders and Challenges**
   - Identify all the stakeholders involved in the system.
   - Explain the roles and responsibilities of each stakeholder.
   - Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach**
   - Describe the existing system or current process.
   - Identify its strengths and limitations.
   - Analyze the problems or gaps that need to be addressed.
4. **Propose a Suitable Solution Using Software Engineering Principles**
   - Design a software-based solution for the given problem.
   - Describe the major functional modules of the proposed system.
   - Identify the functional and non-functional requirements.
   - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
5. **Justify the Chosen Methodology/Framework**

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. **Discuss Expected Outcomes and Limitations**
   - Describe the expected benefits of the proposed solution.
   - Explain how the solution addresses the identified challenges.
   - Discuss potential risks, implementation challenges, and limitations.
   - Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/frame work and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

AI-GENERATED CODE QUALITY AND SECURITY ASSURANCE PLATFORM

1. Understand the Problem Statement

1.1 Problem Statement

AI coding assistants help developers create software quickly by generating code automatically. However, the generated code may contain programming errors, security vulnerabilities, poor coding practices, unnecessary dependencies, and maintenance problems. Developers may not always identify these issues during normal code review. Traditional development processes may also use different tools for code quality, security testing, dependency checking, and automated testing. These tools may not be properly connected with the AI-generated code workflow. Therefore, there is a need for a software platform that automatically checks AI-assisted code before it is merged or deployed. The platform should perform code quality analysis, security validation, dependency checking, and automated testing. It should also identify high-risk changes and send them for human approval. The system must maintain an audit trail so that AI-assisted changes can be traced. This helps organizations use AI coding tools while maintaining software quality, security, and accountability.

1.2 Objectives

1. To analyse AI-generated and AI-assisted source code for quality and security issues.
2. To identify programming errors, insecure coding practices, and code smells.
3. To detect vulnerable, outdated, or unnecessary software dependencies.
4. To integrate automated testing into the development and CI/CD process.
5. To classify code changes according to their security and quality risk.
6. To require human approval for high-risk code changes.

7. To maintain traceability of AI-assisted modifications.
8. To provide reports and dashboards for developers, security teams, and managers.
9. To maintain audit records of code analysis, testing, approvals, and deployment decisions.
10. To support responsible and secure use of AI in software development.

1.3 Scope

The proposed system focuses on analysing software code that is created or modified with the help of AI coding assistants. It can integrate with source-code repositories and CI/CD pipelines. The system performs static code analysis, dependency scanning, security checks, and automated testing. It can classify changes into different risk levels based on the identified problems. High-risk changes can be sent to authorized human reviewers before deployment. The system also maintains audit records of AI-assisted changes and review decisions. A dashboard can provide information about vulnerabilities, failed tests, code quality issues, and pending approvals. Containerized environments can be used to execute tests safely. The system can be designed to support multiple programming languages and development environments. Its scope is to assist developers and security teams rather than completely replace human judgement.

---

2. Identify Key Stakeholders and Challenges

2.1 Key Stakeholders

Software Developers: Developers use AI coding assistants and are responsible for creating, modifying, testing, and fixing source code. They use the platform to identify problems before submitting code for merging.

Security Team: The security team checks vulnerabilities, insecure coding practices, secrets, and dependency risks. They define security policies and investigate serious findings.

Code Reviewers: Reviewers examine the generated or modified code and decide whether it is suitable for merging. They pay special attention to high-risk changes.

DevOps/CI/CD Team: This team integrates the platform with build, testing, and deployment pipelines. They ensure that security and quality checks are executed automatically.

Project Managers: Project managers monitor development progress, quality, risks, and delivery schedules. They can use reports from the platform to understand project status.

System Administrators: Administrators manage system configuration, access permissions, infrastructure, containers, and platform availability.

Management: Organization management uses reports and audit information to understand the benefits and risks of AI-assisted development.

2.2 Major Challenges

1. AI-generated code may contain hidden programming errors.
2. Generated code may introduce security vulnerabilities.
3. AI may suggest vulnerable or outdated dependencies.
4. Large amounts of generated code increase the review workload.
5. Developers may not always understand every generated code segment.
6. Existing development tools may be separate and difficult to integrate.

7. Identifying which changes were AI-assisted can be difficult.
8. Automated security tools may generate false positives.
9. Some vulnerabilities may not be detected automatically.
10. The platform must provide strong security without slowing down development significantly.

---

3. Analyze Current Approach

3.1 Existing System

In the current approach, developers commonly use AI coding assistants to generate or modify source code. The developer then checks the generated code manually before committing it to a version-control system. Code review may be performed by another developer or team member. Automated tests may be executed through a CI/CD pipeline. Separate tools may be used for static analysis, dependency checking, and security scanning. However, these tools may not be specifically connected to AI-assisted development. AI-generated changes may also not be clearly recorded or tracked. High-risk AI-generated modifications may therefore pass through the normal development process without additional verification. Audit information may be distributed across different tools. A centralized platform is therefore needed to combine these activities.

Current process:

AI Code Generation → Developer Review → Version Control → Code Review → Testing → Deployment

3.2 Strengths

1. Developers can use familiar AI coding and development tools.
2. Version-control systems provide a history of code changes.
3. Manual code review provides human judgement.
4. Automated testing can detect some programming errors.
5. Static analysis tools can identify common code-quality problems.
6. Security scanners can detect known vulnerabilities.
7. CI/CD pipelines can automate build and testing activities.
8. Developers can fix problems before deployment.

9. Existing tools provide a foundation for the proposed platform.
10. Human involvement provides additional protection for important changes.

3.3 Limitations and Gaps

1. Existing tools may not specifically identify AI-generated code.
2. Security, quality, dependency, and testing tools may work separately.
3. There may be no centralized view of all analysis results.
4. AI-assisted changes may not have complete traceability.
5. Manual review becomes difficult when AI generates large amounts of code.
6. Vulnerable dependencies may remain unnoticed.
7. Security checks may happen too late in the development process.
8. High-risk changes may not automatically require additional approval.
9. Audit information may be spread across different systems.
10. There may be no unified risk assessment before deployment.

---

4. Propose Suitable Solution Using Software Engineering Principles

4.1 Proposed Solution

The proposed solution is an **AI-Generated Code Quality and Security Assurance Platform** that automatically checks AI-assisted code before it is merged or deployed. Developers submit their code through the normal version-control process. The platform analyses the submitted changes using static analysis, dependency scanning, security validation, and automated testing. The results are combined to calculate the risk level of the change. Low-risk changes can continue through the normal CI/CD process, while high-risk changes can require human approval. The platform records the analysis results, test results, reviewer decisions, and AI-assisted change information. It can provide dashboards and reports for developers, security teams, and management. Containerized environments can be used to safely execute tests. The platform can therefore connect AI-assisted development with quality, security, testing, and governance controls.

Proposed flow:

AI Code → Repository → Code Analysis → Dependency Scan → Security Check → Automated Testing → Risk Assessment → Human Approval if Required → CI/CD → Deployment → Audit

4.2 Major Functional Modules

1. **Code Submission Module** – receives code changes from source-code repositories.
2. **Static Analysis Module** – checks programming errors, code smells, duplication, and coding standards.
3. **Dependency Analysis Module** – checks dependencies for known vulnerabilities and outdated versions.
4. **Security Validation Module** – detects insecure coding practices, secrets, and common security vulnerabilities.
5. **Automated Testing Module** – executes unit, integration, regression, and security tests.
6. **Risk Assessment Module** – calculates the overall risk of a code change.

7. **Human Review Module** – sends high-risk changes to authorized reviewers.
8. **CI/CD Integration Module** – connects analysis results with build and deployment pipelines.
9. **Audit and Traceability Module** – records AI-assisted changes, analysis results, approvals, and deployment decisions.

4.3 Requirements

Functional Requirements

1. The system shall accept code changes from source-code repositories.
2. The system shall identify and record AI-assisted modifications when such information is available.
3. The system shall perform static code analysis.
4. The system shall scan software dependencies for vulnerabilities.
5. The system shall perform security validation.
6. The system shall execute automated tests.
7. The system shall calculate the risk level of code changes.
8. The system shall identify high-risk changes.
9. The system shall support human approval for high-risk changes.
10. The system shall integrate with CI/CD pipelines.
11. The system shall prevent deployment when mandatory checks fail.
12. The system shall maintain audit logs.
13. The system shall provide dashboards and reports.
14. The system shall support role-based access.
15. The system shall maintain traceability of important development activities.

Non-Functional Requirements

The system should provide strong **security** by protecting source code, credentials, analysis results, and audit records. It should provide **reliability** so that analysis and testing results are available consistently. It should provide good **performance** so that security and quality checks do not unnecessarily delay development. The system should be **scalable** so that it can handle increasing numbers of projects, users, repositories, and code-analysis requests. It should be **maintainable** so that new programming languages, security rules, and analysis tools can be

added easily. It should also be **usable**, providing a simple dashboard and understandable reports for developers and reviewers.

4.4 Software Engineering Principles Used

Modularity: The platform is divided into independent modules such as code analysis, dependency scanning, testing, risk assessment, and reporting. This makes development and maintenance easier.

Scalability: The system should support increasing projects, users, repositories, and analysis jobs without major performance problems.

Maintainability: Well-structured modules, documentation, testing, and clear interfaces make future modifications easier.

Reliability: Automated validation, error handling, logging, and repeated testing help provide consistent results.

Usability: Developers should receive clear findings, severity levels, test results, and recommended actions through a simple interface.

Security: Source code, credentials, user accounts, containers, communication, and audit information must be protected from unauthorized access.

---

5. Justify Chosen Methodology/Framework

5.1 Selected Methodology – Agile with DevOps

Agile with DevOps is suitable for developing this platform because the system contains several modules that can be developed and tested incrementally. AI technology and security requirements can change over time, so a flexible development approach is useful. Agile allows the team to divide the project into smaller iterations or sprints. Each sprint can deliver and test a specific part of the system. Feedback from developers and security teams can be used in later sprints. DevOps complements Agile by connecting development, testing, security, and deployment. CI/CD can automatically execute the platform's checks whenever code changes are submitted. Security can also be included throughout the development process rather than being performed only at the end.

5.2 Justification

1. Agile supports incremental development.
2. Requirements can be changed when new security needs appear.
3. Regular feedback improves the platform.
4. Individual modules can be developed and tested separately.
5. Continuous testing helps identify problems early.
6. DevOps supports automated CI/CD integration.
7. Security checks can be included directly in development pipelines.
8. Frequent releases make testing new features easier.
9. Teams can respond quickly to changing AI technologies.
10. Agile with DevOps supports continuous improvement and maintenance.

5.3 Development Process

The project can be developed through multiple iterations. The first iteration can focus on requirements and repository integration. The next iteration can implement static code analysis and dependency scanning. Security validation and automated testing can then be added. After that, risk assessment and human approval can be implemented. CI/CD integration, audit logging,

dashboards, and reporting can be added in later iterations. Each iteration should include development, testing, security verification, and review. Feedback from developers and security teams should be considered before starting the next iteration. This approach reduces development risk and allows the platform to improve continuously.

---

6. Discuss Expected Outcomes and Limitations

6.1 Expected Outcomes

1. The platform can reduce defects in AI-assisted software development.
2. Security vulnerabilities can be identified earlier.
3. Vulnerable software dependencies can be detected before deployment.
4. Automated testing can improve confidence in generated code.
5. Developers can receive faster feedback about code problems.
6. High-risk changes can receive additional human review.
7. AI-assisted modifications can become more traceable.
8. Audit records can improve accountability.
9. CI/CD integration can make quality and security checks more consistent.
10. The system can support responsible adoption of AI coding assistants.

6.2 How Challenges Are Addressed

Challenge	Proposed Solution	Expected Result
AI-generated errors	Static analysis and testing	Earlier error detection
Security vulnerabilities	Security validation	Improved security
Vulnerable dependencies	Dependency scanning	Reduced dependency risk
Large amounts of generated code	Automated analysis	Reduced manual effort
High-risk modifications	Risk assessment and human approval	Better human oversight

Lack of traceability	Audit and tracking module	Improved accountability
Manual testing	CI/CD automation	Faster verification
Unsafe test execution	Containerized execution	Safer testing
Difficult monitoring	Dashboard and reports	Better visibility

6.3 Limitations

1. The platform cannot guarantee that every software defect will be detected.
2. Automated security tools may produce false-positive results.
3. Some vulnerabilities may remain undetected.
4. AI-generated code may not always be identifiable with complete accuracy.
5. Different programming languages may require different analysis tools.
6. Automated tests are limited by the quality and coverage of available test cases.
7. Running multiple analysis and testing tools may require significant computing resources.
8. Integration with existing development environments can be complex.
9. Human judgement is still necessary for important security and architectural decisions.
10. The platform improves software assurance but cannot guarantee completely secure software.

6.4 Future Enhancements

Future versions can include AI-based prediction of high-risk code changes and automatic generation of additional test cases. The platform can provide AI-assisted suggestions for fixing detected vulnerabilities while allowing developers to review the proposed fixes. Software supply-chain security features such as Software Bills of Materials (SBOM) can also be added. Advanced compliance and audit reports can help organizations satisfy regulatory requirements. More programming languages and development platforms can be supported. The system can also learn from previous findings to improve risk classification. Integration with more source-control and CI/CD platforms can increase usability. Advanced dashboards can provide long-term quality and security trends. These enhancements can make the platform more intelligent, secure, and useful for large-scale software development.

6.5 Conclusion

The AI-Generated Code Quality and Security Assurance Platform provides a structured way to control the risks associated with AI-assisted software development. It combines static analysis, dependency scanning, security validation, automated testing, risk assessment, human review, CI/CD integration, and audit tracking into one platform. The proposed system can reduce defects, improve security, increase traceability, and support responsible use of AI coding assistants. Agile with DevOps is suitable because it supports incremental development, continuous testing, security integration, and frequent feedback. However, automated assurance cannot completely replace human judgement or guarantee defect-free and secure software. With future enhancements such as AI-based risk prediction, automatic test generation, and advanced supply-chain security, the platform can provide stronger assurance for modern AI-assisted software development.